PROYECTO DE PROGRAMACIÓN

LA SUPER CARRERA

Memoria técnica del videojuego de carreras en Java

Alumno	Alvaro Martin Lopez
Grupo	Pendiente de completar
Ciclo / curso	1.º DAW
Módulo	Programación
Curso académico	2026/2027
Repositorio GitHub	https://github.com/AlvaroML19/Proyecto-Juego-Java

Simulación de una carrera por consola: estrategia de neumáticos, tiempos por vuelta y clasificación final.

01 · PRESENTACIÓN

Índice y ficha del proyecto

Esta memoria explica el funcionamiento de La Super Carrera a partir de los dos materiales entregados: el README del repositorio y el archivo JuegoCarrera.java. La descripción técnica se ajusta al código disponible; cuando una característica aparece como futura o no está implementada, se indica expresamente.

Contenido

- 1. Presentación y ficha del proyecto
- 2. Descripción del juego: objetivo, reglas y partida
- 3. Simulación de carrera y cálculo de tiempos
- 4. Funcionalidades implementadas y diferencias
- 5. Diseño del programa: variables, constantes y control
- 6. Diseño de datos: explicación de los arrays
- 7. Algoritmo y flujo de ejecución
- 8. Pruebas realizadas y resultados
- 9. Dificultades, mejoras y ampliaciones futuras

Ficha técnica

Elemento	Descripción
Nombre del proyecto	La Super Carrera
Tipo de aplicación	Videojuego de carreras simulado por consola
Lenguaje	Java
Clase principal	JuegoCarrera
Punto de entrada	main(String[] args)
Entrada de usuario	BufferedReader e InputStreamReader
Aleatoriedad	java.util.Random
Repositorio	https://github.com/AlvaroML19/Proyecto-Juego-Java

Alcance de esta memoria Se analiza la versión del código adjunta. No se han añadido funciones que no aparecen en JuegoCarrera.java ni se presupone que las opciones anunciadas para el futuro estén terminadas.

Material base: README.md (descripción y propuesta) y JuegoCarrera.java (implementación analizada y utilizada para las pruebas).

02 · OBJETIVO Y REGLAS

Descripción del juego

La Super Carrera es un videojuego de carreras simulado por consola. El jugador participa en una parrilla formada por seis pilotos. Antes de empezar, introduce su nombre, elige el piloto que quiere controlar, selecciona un compuesto de neumáticos y decide la duración de la carrera, que puede ser de entre 5 y 10 vueltas.

Objetivo

El objetivo es conseguir el menor tiempo total posible y terminar por delante de los demás participantes. El programa calcula el tiempo acumulado de cada piloto y determina como ganador al que tenga el valor más bajo al finalizar todas las vueltas.

Participantes

Piloto	Piloto	Piloto
Fernando Alonso	Carlos Sainz	Max Verstappen
Lewis Hamilton	Lando Norris	George Russell

Reglas principales

1. El jugador escribe su nombre al iniciar el programa. El nombre se utiliza en el mensaje de bienvenida y en la despedida.
2. En el menú, la opción 1 inicia una carrera, la opción 2 muestra el apartado de próximos circuitos y la opción 3 finaliza el programa.
3. Para competir, el jugador selecciona un piloto de la lista, un neumático entre blando, medio y duro, y un número de vueltas entre 5 y 10.
4. Los cinco participantes restantes reciben un tipo de neumático aleatorio. El compuesto elegido por el jugador se asigna al piloto seleccionado.
5. En cada vuelta se calcula un tiempo para cada piloto, se actualiza el desgaste y se muestra quién ocupa el liderato provisional.
6. Al terminar, se presenta una tabla con piloto, neumático y tiempo total. El menor tiempo determina el ganador.

Condición de derrota El código no incluye una pantalla o estado formal de derrota. Si gana otro piloto, simplemente se muestra su nombre como ganador; el jugador puede quedar en otra posición de la clasificación.

03 · MECÁNICA INTERNA

Simulación de carrera y cálculos

La carrera se desarrolla vuelta a vuelta. Para cada piloto, el programa consulta su compuesto, genera un tiempo base aleatorio, aplica la ventaja del neumático y suma una penalización relacionada con el desgaste acumulado. El resultado se añade al tiempo total del piloto.

Comportamiento de los neumáticos

Compuesto	Ventaja por vuelta	Desgaste por vuelta	Característica
Blando	-2,0 s	10,0 puntos	Más rápido al principio, pero se degrada mucho.
Medio	-1,0 s	5,0 puntos	Equilibrio entre rendimiento y duración.
Duro	0,0 s	1,0 punto	No recibe bonificación, pero se desgasta poco.

Cálculo del tiempo de vuelta

El tiempo base se obtiene con la expresión 80 + random.nextDouble() * 10. Por tanto, se encuentra entre 80 segundos y menos de 90 segundos. A continuación, se suma la ventaja del neumático y la penalización por el desgaste que el coche ya tenía acumulado.

```
tiempoVuelta = 80 + random.nextDouble() * 10;
tiempoVuelta += ventajaNeumatico[n];
tiempoVuelta += desgaste[i] * PENALIZACION;
tiempos[i] += tiempoVuelta;
```

La constante PENALIZACION vale 0,06. Por ejemplo, con 20 puntos de desgaste acumulado, la penalización de esa vuelta es 20 × 0,06 = 1,2 segundos. Después de calcular el tiempo, se incrementa el desgaste según el compuesto utilizado.

Límite de desgaste

```
desgaste[i] += degradacionPorVuelta[n];
if (desgaste[i] > 100) {
   desgaste[i] = 100;
}
```

El desgaste queda limitado a 100 puntos. El tiempo de la vuelta utiliza el desgaste anterior al incremento de esa misma vuelta. La actualización del desgaste afecta, por tanto, a la penalización de las vueltas posteriores.

04 · ALCANCE Y COMPARACIÓN

Funcionalidades implementadas

Funciones que sí aparecen en el código

- Menú principal repetitivo que permite iniciar carreras, consultar el apartado de próximos circuitos o salir.
- Personalización de la bienvenida y la despedida mediante el nombre introducido por el jugador.
- Selección de uno de los seis pilotos disponibles.
- Selección de uno de los tres compuestos de neumáticos.
- Elección de una carrera de entre 5 y 10 vueltas.
- Asignación aleatoria de neumáticos a los rivales mediante Random.
- Cálculo aleatorio de tiempos por vuelta, ventajas de neumáticos y penalización por desgaste.
- Actualización del liderato en cada vuelta y presentación de una clasificación final.
- Colores de terminal mediante códigos ANSI y pausa de 700 milisegundos entre vueltas.

Comparación con la propuesta del README

Aspecto descrito	Situación en el código
Nueva carrera con selección de piloto, neumáticos y vueltas	Implementado.
Carrera de 5 a 10 vueltas	Implementado y comprobado con 5 y 10 vueltas.
Rivales con neumáticos aleatorios	Implementado. Cada piloto recibe un índice aleatorio y después se sustituye el del jugador por su elección.
Informe del líder durante la carrera	Implementado en cada vuelta.
Próximos circuitos	No implementado: únicamente muestra «¡Proximamente!».
Gestión de hilos	La descripción del README no es precisa: Thread.sleep(700) pausa el hilo principal; no se crean varios hilos.

Aclaración técnica El README llama «dinámicos» a algunos arrays de estado. En el código son arrays de tamaño fijo, creados una sola vez con la longitud de pilotos y reutilizados en cada carrera.

05 · VARIABLES, CONSTANTES Y ESTRUCTURAS

Diseño del programa

La versión analizada concentra la lógica en la clase JuegoCarrera y dentro del método main. Es un diseño procedimental: las instrucciones se ejecutan de forma secuencial y se apoyan en condicionales, bucles y arrays. No se definen clases propias para representar pilotos, coches o carreras.

Variables principales

Variable	Tipo	Uso
nombreUsuario	String	Guarda el nombre introducido al comenzar.
opcion	int	Almacena la opción elegida en el menú.
pilotoElegido	int	Guarda la posición seleccionada en la lista de pilotos (1 a 6).
neumaticoElegido	int	Guarda la opción de neumático seleccionada (1 a 3).
vueltas	int	Número de vueltas de la carrera.
random	Random	Genera los valores aleatorios de la simulación.
tiempoVuelta	double	Tiempo calculado para un piloto en una vuelta.
n, i, v, lider, mio, ganador	int	Índices y contadores usados en los bucles y búsquedas.

Constantes

Constante	Valor / contenido	Finalidad
PENALIZACION	0.06	Convierte los puntos de desgaste en segundos añadidos.
VERDE, CIAN, AMARILLO, ROJO	Códigos ANSI	Colorean mensajes y resaltan estados en la terminal.
RESET	Código ANSI de reinicio	Devuelve el color de texto al valor normal.

Estructuras de control

El do-while mantiene abierto el menú hasta que opcion sea 3. Los if / else if deciden qué acción ejecutar. Los for recorren la lista de pilotos, inicializan datos, simulan cada vuelta y construyen los resultados. El continue se utiliza cuando una selección numérica está fuera de rango: salta al siguiente ciclo del menú principal.

06 · ESTRUCTURA Y USO DETALLADO

Diseño de datos: arrays

Los arrays permiten almacenar información de varios pilotos o neumáticos sin crear una variable distinta para cada elemento. Los índices comienzan en 0. Por eso, cuando el jugador introduce una opción que comienza en 1, el programa resta 1 para acceder a la posición correspondiente.

Arrays de configuración

Array	Tipo y tamaño	Contenido / función
pilotos	String[6]	Nombres de los seis participantes.
neumaticos	String[3]	Nombres de los tres compuestos.
ventajaNeumatico	double[3]	Bonificación temporal asociada a cada compuesto: -2.0, -1.0 y 0.0.
degradacionPorVuelta	double[3]	Desgaste por vuelta: 10.0, 5.0 y 1.0.

Los arrays ventajaNeumatico y degradacionPorVuelta están relacionados por posición con neumaticos. El índice 0 representa siempre el neumático blando, el 1 el medio y el 2 el duro. Mantener la misma correspondencia es esencial para aplicar los valores correctos.

Arrays de estado de la carrera

Array	Tipo y tamaño	Qué guarda en cada posición
tiempos	double[pilotos.length]	Tiempo total acumulado por cada piloto.
desgaste	double[pilotos.length]	Desgaste actual del neumático de cada piloto.
neumaticoPiloto	int[pilotos.length]	Índice del compuesto montado por cada piloto.

Relación entre posiciones

En los tres arrays de estado, la posición i identifica al mismo piloto. Por ejemplo, tiempos[2], desgaste[2] y neumaticoPiloto[2] describen el tiempo, el desgaste y el neumático del piloto situado en pilotos[2], que es Max Verstappen. Esto permite recorrer todos los participantes usando el mismo índice.

```
for (int i = 0; i < pilotos.length; i++) {
   tiempos[i] = 0;
   desgaste[i] = 0;
   neumaticoPiloto[i] = random.nextInt(neumaticos.length);
}
neumaticoPiloto[pilotoElegido - 1] = neumaticoElegido - 1;
```

Al comenzar una carrera válida se ponen a cero tiempos y desgaste. Después, se asigna un compuesto aleatorio a cada piloto y se reemplaza el del jugador por el que ha elegido. Los arrays se reutilizan en una nueva carrera, pero se reinicializan antes de empezar sus vueltas.

07 · LÓGICA DEL PROGRAMA

Algoritmo y flujo de ejecución

Secuencia general

Etapa	Acción
INICIO	Crear lector, variables, constantes y arrays.
IDENTIFICACIÓN	Leer el nombre y mostrar bienvenida.
MENÚ	Solicitar una opción mientras no sea 3.
CONFIGURACIÓN	Si se elige nueva carrera: piloto, neumático y vueltas.
VALIDACIÓN	Comprobar que piloto está entre 1 y 6, neumático entre 1 y 3 y vueltas entre 5 y 10.
PREPARACIÓN	Reiniciar los arrays de estado y asignar neumáticos.
CARRERA	Calcular vueltas, desgaste, tiempos y líder.
CLASIFICACIÓN	Buscar el menor tiempo y mostrar los resultados.
FIN	Al elegir 3, despedir al jugador y terminar.

Búsqueda del líder y del ganador

El código utiliza una búsqueda lineal del menor tiempo acumulado. Primero supone que el piloto en la posición 0 va en cabeza. Después recorre las demás posiciones. Si encuentra un tiempo inferior al del líder actual, actualiza el índice lider. El mismo procedimiento se repite al final para obtener ganador.

```
int lider = 0;
for (int i = 1; i < pilotos.length; i++) {
    if (tiempos[i] < tiempos[lider]) {
        lider = i;
    }
}
```

El algoritmo es sencillo y suficiente para seis pilotos: comprueba cada participante una vez por búsqueda. Se emplea una comparación estricta (<), porque el programa busca el menor tiempo total.

Pseudocódigo simplificado de una vuelta

```
PARA cada vuelta:
    PARA cada piloto:
        consultar su neumático
        generar tiempo base aleatorio
        sumar ventaja del neumático
        sumar penalización por desgaste acumulado
        acumular tiempo total
        aumentar desgaste y limitarlo a 100
    buscar piloto con menor tiempo acumulado
    mostrar vuelta y líder
    pausar 700 milisegundos
```

08 · VERIFICACIÓN

Pruebas realizadas y resultados

Se compiló el archivo JuegoCarrera.java con Java 21 y se ejecutaron pruebas desde la consola mediante entradas preparadas. Como los tiempos dependen de Random, los valores exactos cambian entre ejecuciones; por ello, la comprobación se centra en que el flujo, los límites y los mensajes respondan correctamente.

Caso	Entrada / escenario	Resultado observado
Compilación	javac JuegoCarrera.java	Compilación correcta, sin errores.
Salida normal	Nombre y opción 3	Muestra bienvenida, menú y despedida.
Nueva carrera mínima	Piloto 1, neumático 1, 5 vueltas	Completa las cinco vueltas, muestra líder y clasificación.
Nueva carrera máxima	Piloto 6, neumático 3, 10 vueltas	Completa las diez vueltas y muestra ganador.
Opción de menú incorrecta	Opción 8	Muestra «Opcion no valida» y vuelve al menú.
Piloto fuera de rango	Opción 0	Muestra «Piloto no valido» y vuelve al menú.
Neumático fuera de rango	Opción 4	Muestra «Neumatico no valido» y vuelve al menú.
Vueltas por debajo del mínimo	4 vueltas	Muestra «Numero de vueltas no válido» y vuelve al menú.
Texto donde se espera número	abc en el menú	Se produce NumberFormatException; no hay control de este caso.

Comprobación de límites

Se ejecutó una carrera de 5 vueltas y otra de 10 vueltas. Ambas completaron el bucle de simulación y mostraron el resultado final. Los valores 4 y otros valores fuera del intervalo se rechazaron. El código permite también valores intermedios, como 6, 7, 8 o 9, por la condición vueltas < 5 || vueltas > 10.

Observaciones de las pruebas

- Las entradas numéricas incorrectas dentro del rango de lectura se controlan con condicionales.
- Las entradas que no pueden convertirse a entero, como «abc», no se controlan mediante try-catch y provocan la finalización anormal del programa.
- Las pruebas de carrera no producen el mismo ganador en todas las ejecuciones, ya que el tiempo base y los neumáticos de los rivales son aleatorios.

Evidencia La compilación y las ejecuciones de prueba se realizaron sobre el archivo adjunto. Las cifras de tiempos de carrera son variables y no deben interpretarse como resultados fijos.

09 · VALORACIÓN FINAL

Dificultades, soluciones y ampliaciones

Dificultades técnicas identificadas

Dificultad / aspecto	Solución aplicada o estado actual
Gestionar las elecciones numéricas	Se comprueban los rangos permitidos antes de iniciar la carrera.
Relacionar cada neumático con sus parámetros	Se usan arrays paralelos y un índice común para ventaja y degradación.
Controlar el estado de seis pilotos	Los arrays de tiempos, desgaste y neumático comparten el índice del array pilotos.
Determinar quién lidera	Se recorre el array de tiempos y se conserva el índice con el menor valor.
Evitar un desgaste superior a 100	Se limita el valor después de incrementarlo.
Simular el paso del tiempo	Se utiliza Thread.sleep(700) al terminar cada vuelta.

Mejoras recomendables

El programa funciona en los escenarios numéricos previstos, pero hay aspectos que se pueden mejorar. El más importante es controlar entradas no numéricas con un bloque try-catch para evitar que NumberFormatException cierre el juego. También convendría volver a preguntar una selección incorrecta sin obligar al usuario a empezar de nuevo desde el menú.

Ampliaciones para la segunda evaluación

- Implementar distintos circuitos, con nombres y características propias, en lugar del mensaje provisional actual.
- Permitir que la carrera tenga varias sesiones o campeonatos con clasificación acumulada.
- Añadir paradas en boxes y posibilidad de cambiar neumáticos durante la carrera.
- Crear clases Piloto, Coche, Neumatico y Carrera para organizar mejor el código y aplicar programación orientada a objetos.
- Mostrar estadísticas adicionales: mejor vuelta, diferencia entre pilotos, posición del jugador y desgaste restante.
- Guardar resultados de carreras para consultarlos en futuras ejecuciones.
- Mejorar la validación de entradas y los mensajes de error.

Conclusión

La Super Carrera reúne los contenidos básicos de programación trabajados en el proyecto: entrada por consola, variables, constantes, arrays, estructuras condicionales, bucles, generación de números aleatorios y operaciones con datos numéricos. La simulación permite configurar una carrera, observar el liderato vuelta a vuelta y consultar una clasificación final. La siguiente evolución lógica consiste en reforzar la validación de entradas, ampliar las opciones de juego y separar la lógica en métodos y clases.